1. HW05 Summary of Objet Oriented Modeling and Uml

- Conceptual modeling helps understand, visualize, and communicate real-world systems.
- Best practices include organizing software into loosely coupled layers preferring aggregation over inheritance, and programing to an interface or abstraction
- Meaningful identifiers should be used for classes, variables, and methods
- Developers should rely on executable unit tests to find desing errors and verify code correcthess

2. Introduction to UML

- UML is a semi-formal, expressive language used to model object-oriented systems
- Models are viewed from three primary perspectives: Analysis, Specification, and Implementation
- UML diagroms are categorized into Structural diagram and Behavioral diagroms

3. Basic Use Cases

- Actors represent external users, hardware, or other systems that interact with the system to achive a goal
- Use cases capture user-visible functionality, explaining "what" the system does and "why" rather than "how"
- Relationships between use cases include <extend> for variations <include> for required sub-goals, and Generalization for specialized subset

4. Class Diagrams

- Objects are the basic building blocks that encapsulate state and behavior
- Classes are abstractions that group objects with shared characteristics
- Classes are defined by their attributes and operations
- Key structural relationships include Associations, Generalizations Aggregations, Compositions, and Dependencies

5. UML Quick Reference Guide

- During the analysis phase, modeling should focus on real-world problems and stakeholder needs rather than software implementation details
- Activity diagrams map out step-by-step processes using action nodes, control flows forks and joins
- Interaction diagrams model how objects collaborate by passing synchronous or asynchronous messages over time
- State charts illustrate an object's operational states and the transitions triggered by specific events and conditions